Pods, Namespaces e Controladores

Formas de gerenciamento e os objetos de carga de trabalho



Ada · DE-OP-011 Orquestração · Kubernetes | Aula 03 · Prof.: Everton J Benedicto

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada = aula top!

INTERVALO • 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

Objetivos de hoje

- Diferenciar forma Declarativa × Imperativa
- Aprofundar o conceito de Pod e seus status
- Organizar recursos com Namespaces
- Controladores: Deployment, StatefulSet e DaemonSet
- Escolher o controlador certo para cada tipo de aplicação

1. Declarativo × Imperativo

Duas formas de falar com a API

Forma Declarativa

- Descrevemos o ESTADO DESEJADO em arquivos YAML (manifestos)
- YAML: formato legível, próximo da linguagem humana
- O K8S lê o manifesto e faz o necessário para atingir aquele estado
- Versionável em Git, reexecutável, ideal para produção
- Aplicado com: kubectl apply -f arquivo.yaml

Manifesto de um Pod (declarativo)

pod-nginx.yaml

```yaml
apiVersion: v1
kind: Pod              # tipo de recurso
metadata:
  name: nginx          # nome do Pod
spec:
  containers:
  - name: nginx        # nome do container
    image: nginx:1.14.2 # imagem usada
    ports:
    - containerPort: 80 # porta exposta no container
```

Forma Imperativa

- Damos ordens diretas na linha de comando, sem arquivo
- O K8S descobre como executar o que foi pedido
- Prática e rápida para tarefas pontuais e testes
- Exemplo: kubectl run --image=nginx nginx2
- Desvantagem: não fica versionada nem documentada

Comparando as duas formas

MÃO NA MASSA

Declarativo — cria a partir do arquivo:

```
$ kubectl apply -f pod-nginx.yaml
$ kubectl get pods
```

Imperativo — cria direto no comando:

```
$ kubectl run --image=nginx nginx2
```

Inspecione o resultado em YAML:

```
$ kubectl get pod nginx2 -o yaml
```

Comparando as duas formas

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl apply -f aula-03\pod-nginx.yaml
pod/nginx created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          31s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl run --image=nginx nginx2
pod/nginx2 created
```

Fonte da imagem: Ada Tech

Comparando as duas formas (cont.)

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
nginx    1/1     Running   0          2m
nginx2   1/1     Running   0          39s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl get pod nginx2 -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2026-07-06T16:35:12Z"
  generation: 1
  labels:
    run: nginx2
  name: nginx2
  namespace: default
  resourceVersion: "536"
  uid: 8b5ace6d-7a47-46b7-ae30-96aa62aaa7d3
spec:
  containers:
  - image: nginx
    imagePullPolicy: Always
    name: nginx2
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
```

Fonte da imagem: Ada Tech

2. Pods em profundidade

A menor unidade computacional

Prof.: Everton J Benedicto



O que dá para configurar num Pod

- Um ou mais containers e suas respectivas imagens
- Init containers: executam só no "nascimento" do Pod
- requests e limits: recursos disponíveis por container (Aula 5)
- Volumes para armazenamento (Aula 6)
- Containers de um Pod compartilham rede e podem compartilhar volumes

Status do ciclo de vida de um Pod

- Pending — implantado, mas um ou mais containers ainda não disponíveis
- Running — implantado e com um ou mais containers em execução
- Succeeded — todos os containers concluíram com sucesso (comum com init containers)
- Failed — um ou mais containers terminaram em falha
- Terminating — o Pod está sendo removido

3. Namespaces

Organizando recursos no cluster

Prof.: Everton J Benedicto

Namespaces: divisão lógica do cluster

- Agrupam recursos dentro de um mesmo cluster
- Nome de recurso é único DENTRO de um namespace (permite repetir entre namespaces)
- Separam por time, ambiente, tipo de aplicação...
- Padrão: default, kube-system, kube-public, kube-node-lease
- Nem tudo tem namespace: nodes e volumes, por exemplo, não têm

Trabalhando com Namespaces

MÃO NA MASSA

Listar namespaces (forma completa e abreviada):

```
$ kubectl get namespaces
$ kubectl get ns
```

Ver os componentes internos do próprio Kubernetes:

```
$ kubectl get pods -n kube-system
```

Criar um namespace:

```
$ kubectl create ns teste-k8s
```

Consultar recursos de um namespace específico com -n:

```
$ kubectl get pods -n teste-k8s
```

Trabalhando com Namespaces

```
PS C:\> kubectl get namespaces
NAME              STATUS   AGE
default           Active   36m
kube-node-lease   Active   36m
kube-public       Active   36m
kube-system       Active   36m
PS C:\> kubectl get ns
NAME              STATUS   AGE
default           Active   36m
kube-node-lease   Active   36m
kube-public       Active   36m
kube-system       Active   36m
PS C:\> kubectl get pods -n kube-system
NAME                               READY   STATUS    RESTARTS      AGE
coredns-7d764666f9-l7zf4           1/1     Running   0             36m
etcd-minikube                      1/1     Running   0             36m
kube-apiserver-minikube            1/1     Running   0             36m
kube-controller-manager-minikube   1/1     Running   0             36m
kube-proxy-hkf8p                   1/1     Running   0             36m
kube-scheduler-minikube            1/1     Running   0             36m
storage-provisioner                1/1     Running   1 (36m ago)   36m
PS C:\> kubectl create ns teste-k8s
namespace/teste-k8s created
PS C:\> kubectl get pods -n teste-k8s
No resources found in teste-k8s namespace.
```

Fonte da imagem: Ada Tech

4. Controladores

Deployment, StatefulSet e DaemonSet

Prof.: Everton J Benedicto

Por que não criar Pods soltos?

- Pod solto que morre NÃO volta sozinho
- Controladores mantêm o estado desejado (self-healing)
- Definem quantas réplicas, quais imagens, como atualizar
- Três tipos principais: Deployment, StatefulSet, DaemonSet

Deployment (aplicações stateless)

nginx-deployment.yaml — o controlador mais usado

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3            # número de Pods
  selector:
    matchLabels:
      app: nginx         # vincula os Pods ao Deployment
  template:              # o "molde" de cada Pod
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

StatefulSet (aplicações stateful)

- Para aplicações com estado: bancos de dados, filas, etc.
- Cada Pod tem identidade persistente e estável (web-0, web-1, web-2)
- Se um Pod morre, volta com a MESMA identidade
- Permite associar volume persistente por Pod
- Permite ordem controlada de subida dos Pods

DaemonSet (um Pod por node)

- Garante um Pod da aplicação em CADA node do cluster
- Node novo entrou? Um Pod é criado nele automaticamente
- Usos típicos: coleta de logs, monitoração de nodes
- Pode rodar até nos masters (com tolerations — Aula 5)

Qual controlador usar?

- App sem estado (web, API) → Deployment
- App com estado/identidade (banco de dados) → StatefulSet
- Um agente por node (logs, monitoração) → DaemonSet
- A escolha certa simplifica muito a operação do cluster

Recapitulando a Aula 03

- ✓ Declarativo (YAML, produção) × Imperativo (CLI, testes)
- ✓ Pod é a menor unidade; conheça seus status
- ✓ Namespaces organizam e isolam recursos logicamente
- ✓ Deployment / StatefulSet / DaemonSet: um para cada necessidade
- ✓ Próxima aula: subir nossa PRIMEIRA aplicação de verdade!

